

Murray State University

Zero to Unbeatable

Demo 1 - rules, machine learning, neural networks - 20 minutes

You need: one browser and the demo's `index.html`. It works offline. For projection, use Settings -> Presentation mode. Start with a fresh page so the group sees the learning take place.

Purpose

This is not a race from "simple AI" to "better AI." It puts three different sources of a program's skill beside the same small game: a person's written instructions, experience stored in a table, and shared weights that learn to approximate that table.

"Watch where the skill comes from. The board stays the same; the mechanism changes."

1a - Rules-based intelligence - 5 minutes

Open **1a Rules-based intelligence**. Ask a volunteer to play the full eight-rule opponent. Point to the highlighted rule after each reply. It can say why it moved because a person wrote its decision process down in advance.

1. Switch to First 2 and invite the room to find a fork.
2. Return to All 8. Explain that the app checks every reachable game line against this policy. The full rule set has no losing line.
3. Ask: "If it is skilled but never learned, should we call it AI?"

The useful distinction is not whether the program is impressive. It is whether its competence came from a person's rules or from experience.

1b - Machine learning - 7 minutes

Open **1b Machine learning**. Era 0 begins with a table full of zeroes. Let someone beat it, then run a visible training burst and play the new era. The program changes only the numeric values it has learned from completed games.

1. Use the era selector to compare an earlier and later opponent.
2. Turn on Show its brain. Each empty square displays the learned score for the position created by that move.
3. When a later era is checked, distinguish a search-based proof from a good-looking win streak.

"This is machine learning, but its memory is a filing cabinet: one number for each position it has room to store."

1c - Neural networks - 7 minutes

Open **1c Neural networks**. The first button says **Create learning examples**. It visibly lets a fresh copy of the 1b algorithm play 20,000 seeded games, then freezes the values it learned. Those frozen examples are the teacher for a new small network of shared weights.

1. Press Create learning examples. Explain that these are learned examples, not answers copied from a perfect-game solver.
2. Press Train network. The live display shows how closely the weights match examples used for training and positions kept out of training.
3. After a run, read the separate playing score. Each 100-game check uses the same random seed and scoring procedure, though game paths can change as the network changes. MSE means average squared prediction-score error; zero is an exact match. A lower error and a better score are observations, not a promise that the network will handle every unseen position well.
4. Turn on Show its brain. Its squares now come from a network calculation. The frozen table is not consulted while it chooses a move.

The examples are the fresh table's current estimates. A position it never visited can still have its initial score of zero, even where a perfect-game solver would give a different answer.

Technical detail for interested students: rotations and reflections of the same board are kept together when examples are split, preventing an obvious near-copy from appearing in both the practice and check sets. The app's source test also compares its hand-written gradient with a finite-difference estimate away from ReLU corners.

"A neural network is a kind of machine learning. Here it is learning a compact approximation of another learner's experience, not receiving tic-tac-toe strategy by hand."

Optional advanced extension

The Neural networks tab links to Ultimate tic-tac-toe as an optional representation-limit activity. It changes the game and adds global information: a board's place in the meta-grid and where a square sends the opponent. A one-board table leaves those facts out, and a neural network with the same inputs would too. It is deliberately separate from the neural-network lesson.

Discussion prompts

- Which opponent is easiest to explain? Which is easiest to update?
- Why is a held-out score more useful than reporting only the examples used to train?
- What would have to be checked before calling a trained network unbeatable?

Bryant Harrison - Murray State University - Zero to Unbeatable - single-file offline demo - no accounts or runtime network requests